University of Southampton

COMP6237 Data Mining
Lecture 8: Nearest Neighbours

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

Lecturer (Assistant Professor) @ VLC of ECS

University of Southampton

Lecture slides available here:

http://comp6237.ecs.soton.ac.uk/zh.html

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Nearest Neighbours – Problem

Nearest Neighbours (NN) problem: how to use the **nearest neighbours** to make a prediction?

In **item-based recommendation**, we use the same idea but with *items* as points:

- **Each item (movie)** is a point in a feature space.
- A natural feature representation is its **rating vector over users**.
- For Miley, we do the prediction based on the weighted sum of **top-N most similar items**.

	Life is Beautiful	Seven Samurai	Joker	Schindler's List	The Pianist	City Of God
Lee	2.5	3.5	3.0	3.5	3.0	2.5
Sofia	3.0	3.5	1.5	5.0	3.0	3.5
Miley	2.5	3.0	NaN	3.5	4.0	NaN
Justina	NaN	3.5	3.0	4.0	4.5	3.0
Donald	3.0	4.0	2.0	3.0	3.0	2.0
Mickey	3.0	4.0	NaN	5.0	3.0	3.5
Tristan	NaN	4.5	NaN	4.0	NaN	1.0

Nearest Neighbours – KNN Algorithm

University of Southampton

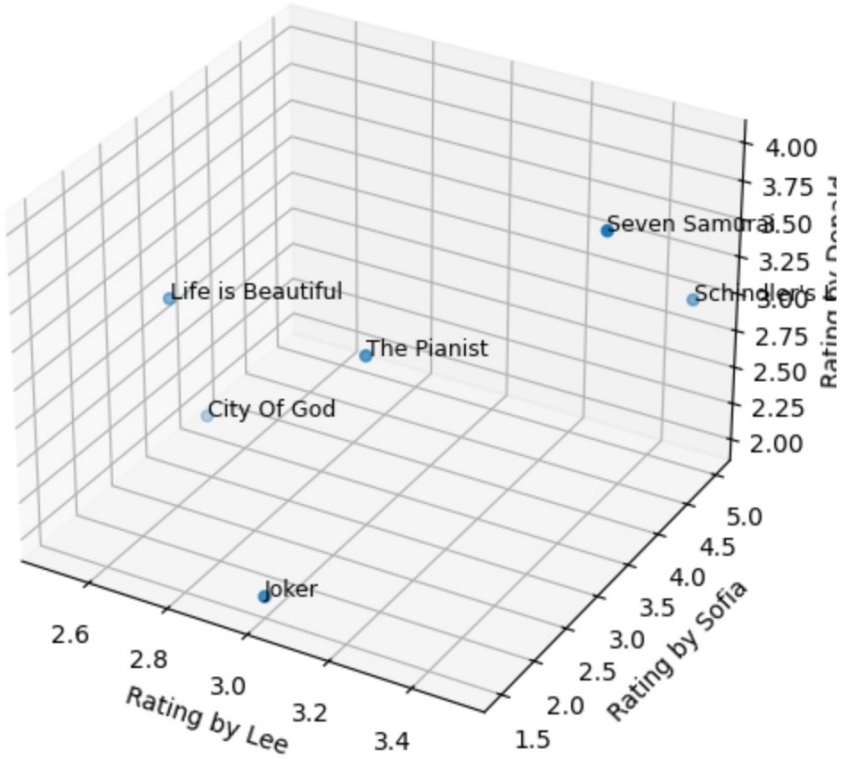
Key idea and "lazy learning"

It uses the K nearest neighbours for each prediction.

We treat **each movie** as a stored "case":

- **Feature vector for a movie** i is its ratings over users:
  $\mathbf{x}_i = [r_{1,i}, r_{2,i}, \ldots, r_{U,i}]$
- "Training" is just **storing the cases** (no model fitting).
- Computation happens at query time: compute distances/similarities and retrieve neighbours.

Below: we visualise movies as points in a **3D user-rating space** using 3 users as axes.



Nearest Neighbours – KNN Algorithm

University of Southampton

The k **nearest neighbours** of a query point $\mathbf{x}$ are the k stored points with the smallest distance:
$\mathcal{N}_k(\mathbf{x}) = \arg\min_{k \text{ points}} \ \text{dist}(\mathbf{x}, \mathbf{x}_i)$.

From neighbours to a decision (two common outputs)

1. **Classification** (majority vote):

$$\hat{y}(\mathbf{x}) = \text{mode}\{y_i : i \in \mathcal{N}_k(\mathbf{x})\}.$$

2. **Regression** (weighted average):

$$\hat{y}(\mathbf{x}) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i \, y_i}{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i}.$$

In our item-based case:

- Query item could be **Joker** (to find similar movies)
- For Miley we can treat each already-rated movie as "labelled":
  - label = **like** if rating ≥ 3.5 else **not like**
- Then do **k-NN classification** ("Will Miley like Joker?") and/or **k-NN regression** ("What rating might she give?").

Nearest Neighbours – KNN Algorithm

University of Southampton

```
Target: Joker  -> predicted label = not like, predicted rating ≈ 3.18
```

	item	similarity	n_common_users	user_rating	user_label
0	The Pianist	0.974893	4	4.0	like
1	Seven Samurai	0.957621	4	3.0	not like
2	Life is Beautiful	0.936013	3	2.5	not like

```
Target: City Of God  -> predicted label = like, predicted rating ≈ 3.33
```

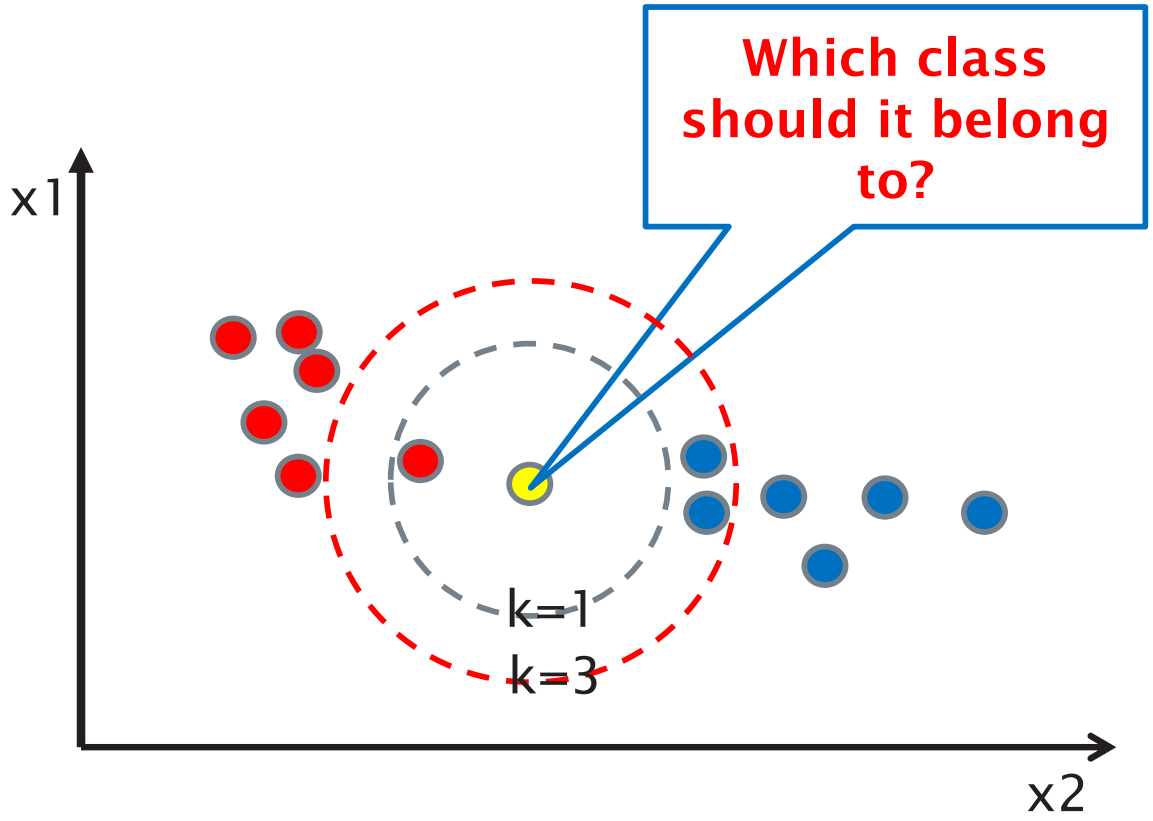
	item	similarity	n_common_users	user_rating	user_label
0	Life is Beautiful	0.978179	4	2.5	not like
1	The Pianist	0.967589	5	4.0	like
2	Schindler's List	0.967284	6	3.5	like

Demo: Nearest_Neighbours_Item_based

Nearest Neighbours – KNN Algorithm

University of Southampton

Different K might have different K-NN classification results



Nearest Neighbours – KNN Algorithm

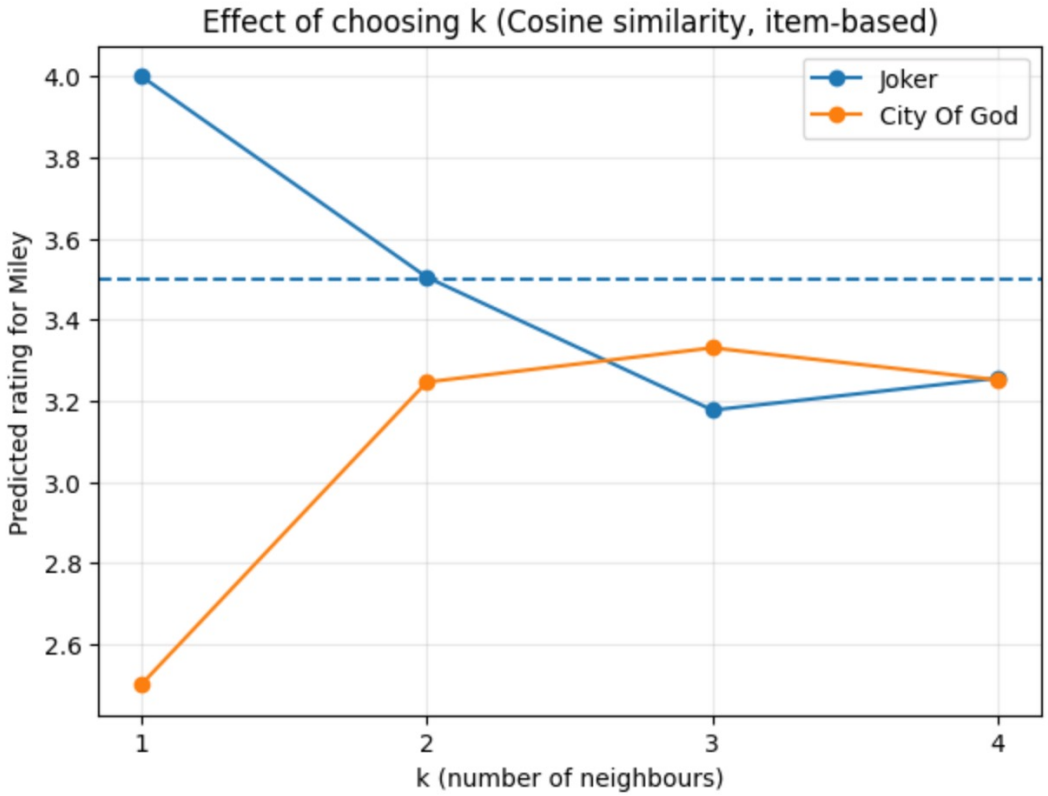
University of Southampton

The same trade-off holds:

- If k is **too small**: prediction is sensitive to noise / quirks of a single neighbour.
- If k is **too large**: neighbours may include dissimilar items, blurring the decision.

We visualise how Miley's predicted ratings change as we vary k.

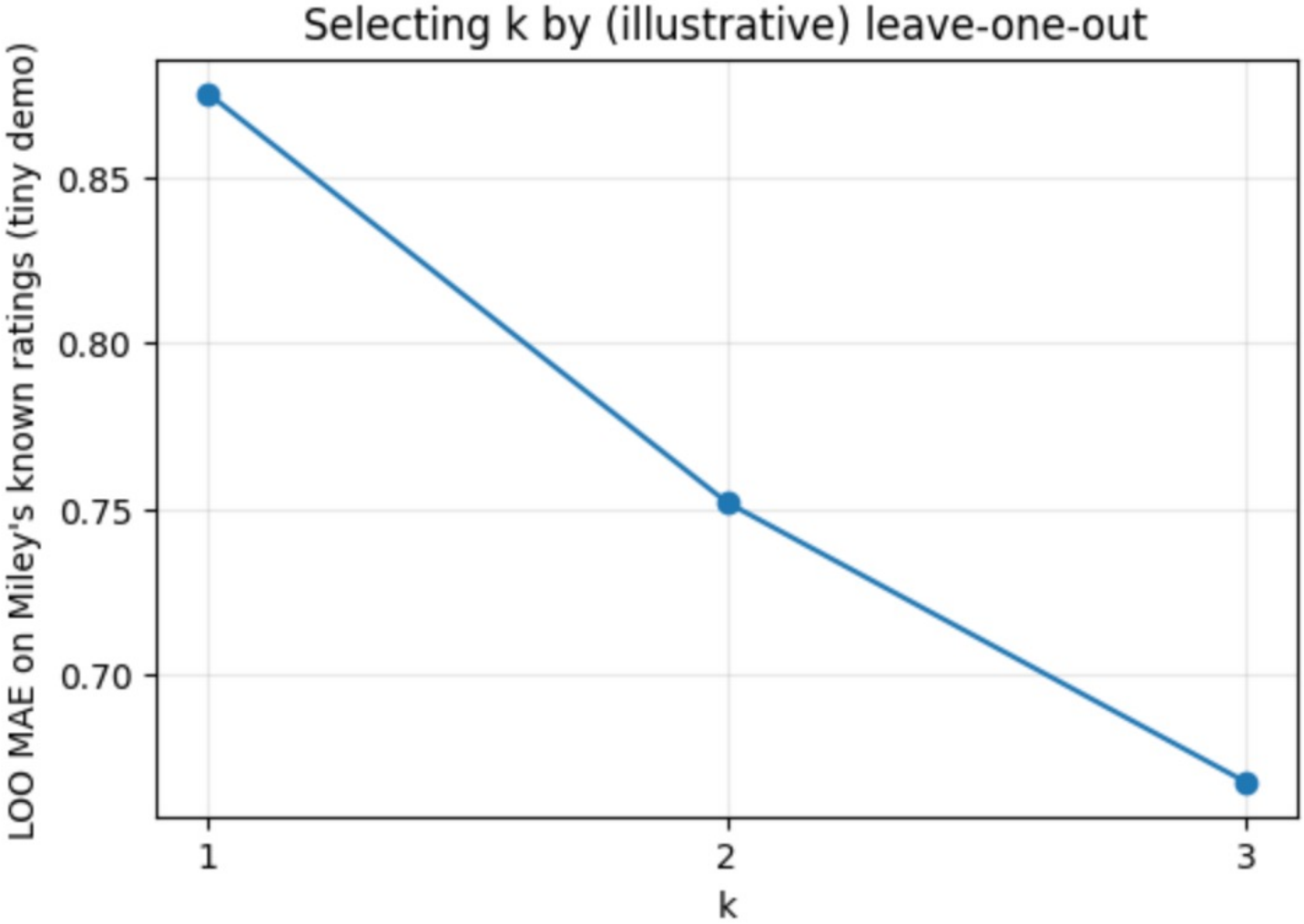
	target_item	k	pred_rating	pred_label
0	Joker	1	4.000000	like
1	Joker	2	3.504469	like
2	Joker	3	3.176706	not like
3	Joker	4	3.255638	like
4	City Of God	1	2.500000	not like
5	City Of God	2	3.245918	like
6	City Of God	3	3.330286	like
7	City Of God	4	3.251074	like



Nearest Neighbours – KNN Algorithm

University of Southampton

In practice we often use **cross-validation** to select k (and a weighting scheme).



Nearest Neighbours – Weighted KNN

University of Southampton

Weighted k-NN classification

Goal: **predict a class label**

Steps:

1. Find the **k nearest neighbours**
2. Compute weights based on distance
3. **Sum the weights for each class**
4. Choose the **class with the largest total weight**

Example

```
Class A weights: 0.7 + 0.4 = 1.1
Class B weights: 0.5 + 0.3 = 0.8
→ predict Class A
```

So classification uses **weighted voting**.

k-NN regression (weighted)

Goal: **predict a numeric value**

Steps:

1. Find the **k nearest neighbours**
2. Compute weights
3. Compute a **weighted average of the target values**

$$\hat{y} = \frac{\sum w_i y_i}{\sum w_i}$$

Example

```
ratings = [4.5, 4.0, 3.0]
weights = [0.6, 0.3, 0.1]
prediction ≈ 4.2
```

So regression uses a **weighted average**.

Nearest Neighbours – Weighted KNN

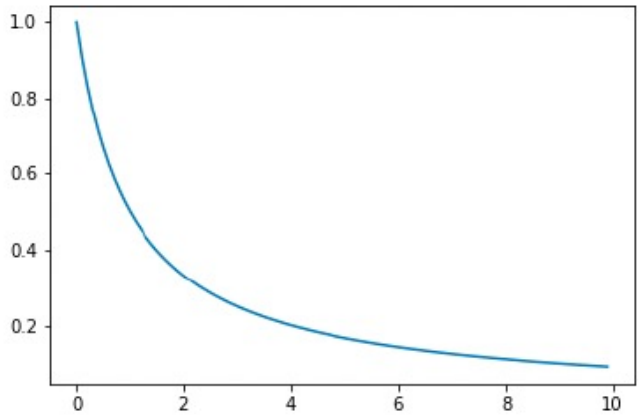
University of Southampton

Weighted k-NN classification

Inverse Weighting:

$$w = \frac{1}{dist + c}$$

Where c is a constant, avoiding division by zero error if $dist = 0$



Nearest Neighbours – Weighted KNN

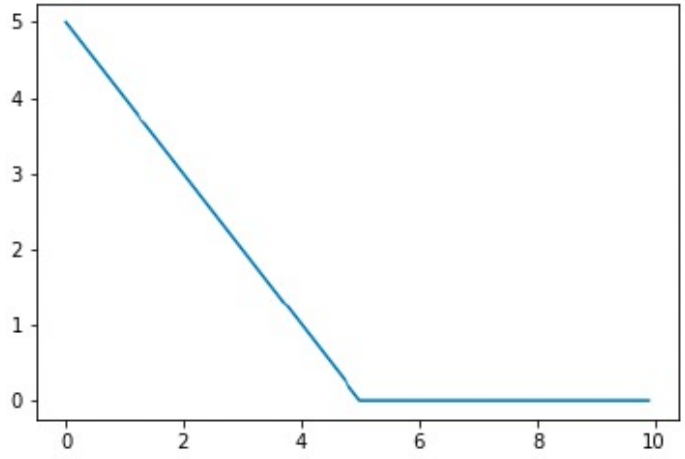
University of Southampton

Weighted k-NN classification

Subtraction Weighting:

$$w = \max(0, c - dist)$$

Where c is a constant



Nearest Neighbours – Weighted KNN

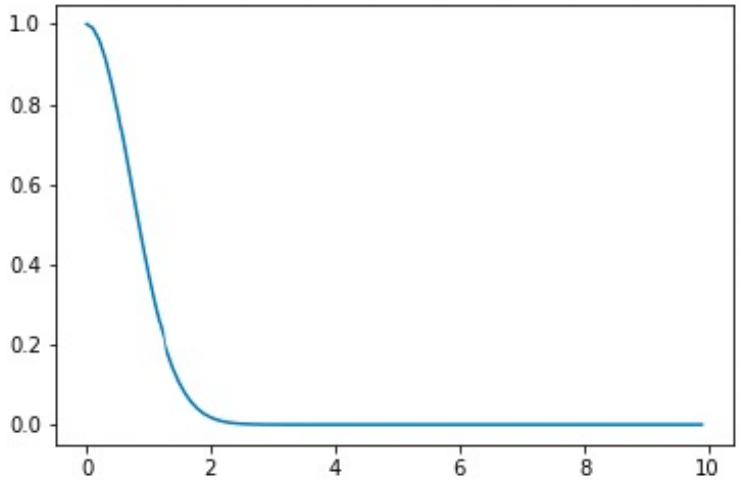
University of Southampton

Weighted k-NN classification

Gaussian Weighting:

$$w = \exp \frac{-dist^2}{c^2}$$

Where c is a constant



Nearest Neighbours – KNN

Advantages?

- No assumptions made
- No training phase
- Simple and easy to implement

Problems?

- Doesn't scale well with lots of data
- Doesn't scale well with many dimensions

Nearest Neighbours – KNN

Curse of dimensionality; For low dimensions, the size of a neighbourhood is small.

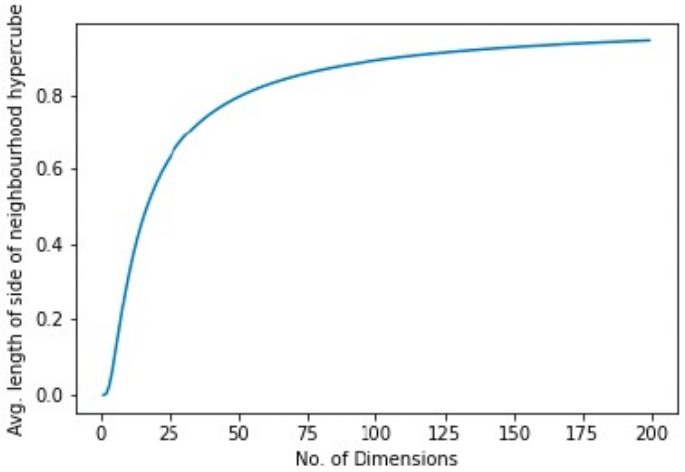
e.g. for $k = 10$, number of points $N = 1,000,000$

In a unit line, the average neighbourhood is $\frac{10}{10^6} = 0.00001$ long

In a unit square, the average side length is $\sqrt{\frac{10}{10^6}} = 0.003$ long

In a unit cube, the average side length is $\sqrt[3]{\frac{10}{10^6}} = 0.02$ long

As the dimensionality increases, the average neighborhood size for k=10 grows rapidly



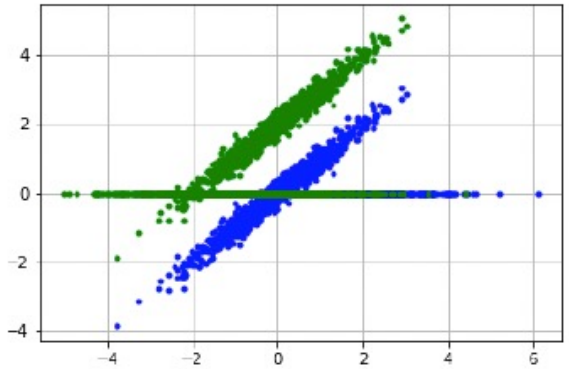
In very high dimensions (like 200), to find the 10 nearest neighbors for a point, you essentially need to consider above 90% of the entire dataset of 1 million points

This can make it very difficult to work out which are closer, as the distances are nearly all the same

Nearest Neighbours – KNN

Solutions for high D?

- Dimensionality reduction.. Care! Some aren't suitable (e.g. MDS, SOM) for extremely high dim
- Also.. PCA:



- A random direction could be better!
  Johnson Lindenstrauss lemma:
  if points in a vector space are of high enough dimensionality, they may be projected into a lower dimensional space in a way which approximately preserves the distances between the points, this basis can be generated randomly

Nearest Neighbours – KNN

University of Southampton

Solutions for lots of data?

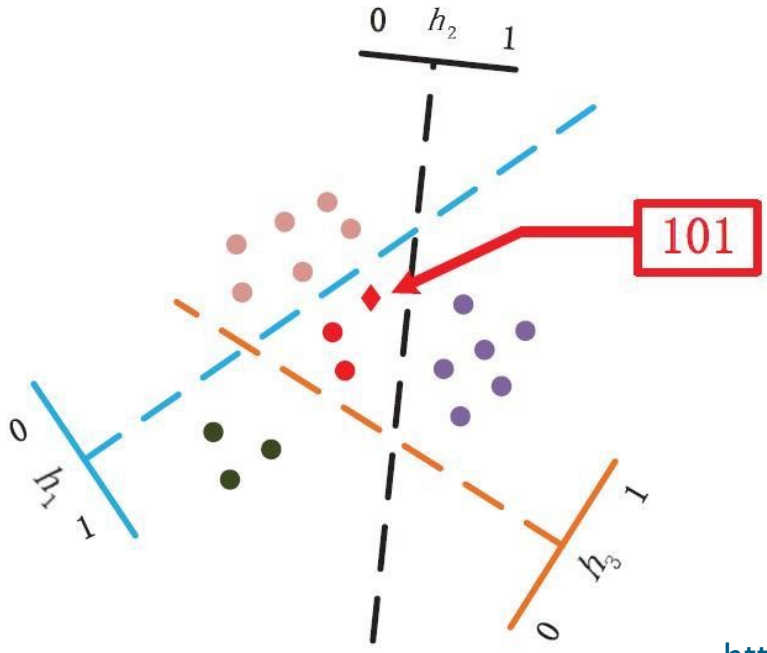
- Need to quickly find the nearest neighbour to a particular point in a highly dimensional space
  - Could index points in a tree structure?
  - Could hash the points?
  - Could break up the space

This lecture we focus on gashing method.

Nearest Neighbours – LSH

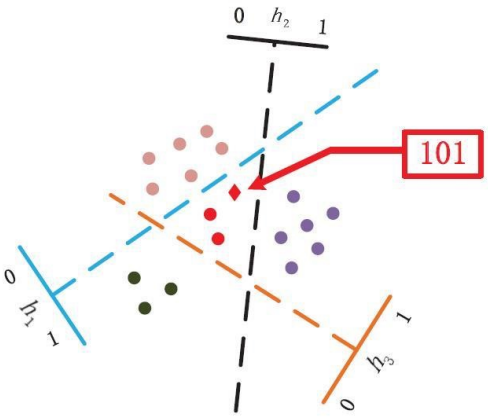
Locality Sensitive Hashing
Makes hash codes that are similar for similar vectors

- Similar items map to the same buckets with high probability
- number of buckets much smaller than number of data samples
- Aims to maximise the probability of a collision for similar items



https://www.researchgate.net

Nearest Neighbours – LSH



Accomplished by:

- Chose random hyperplanes $(h_1, h_2, \ldots, h_k)$
- Each hyperplane with split the space in to 2 regions
- $\therefore$ the space will be sliced in to 2^k regions (buckets)
- Giving a simple code for each of the data points, depending on which side of each hyperplane the data points are
- The same code for each of the data points within the same region
- Compare new point only to training points in the same region
- Repeat with different random hyperplanes $(h_1, h_2, \ldots, h_k)$

on board

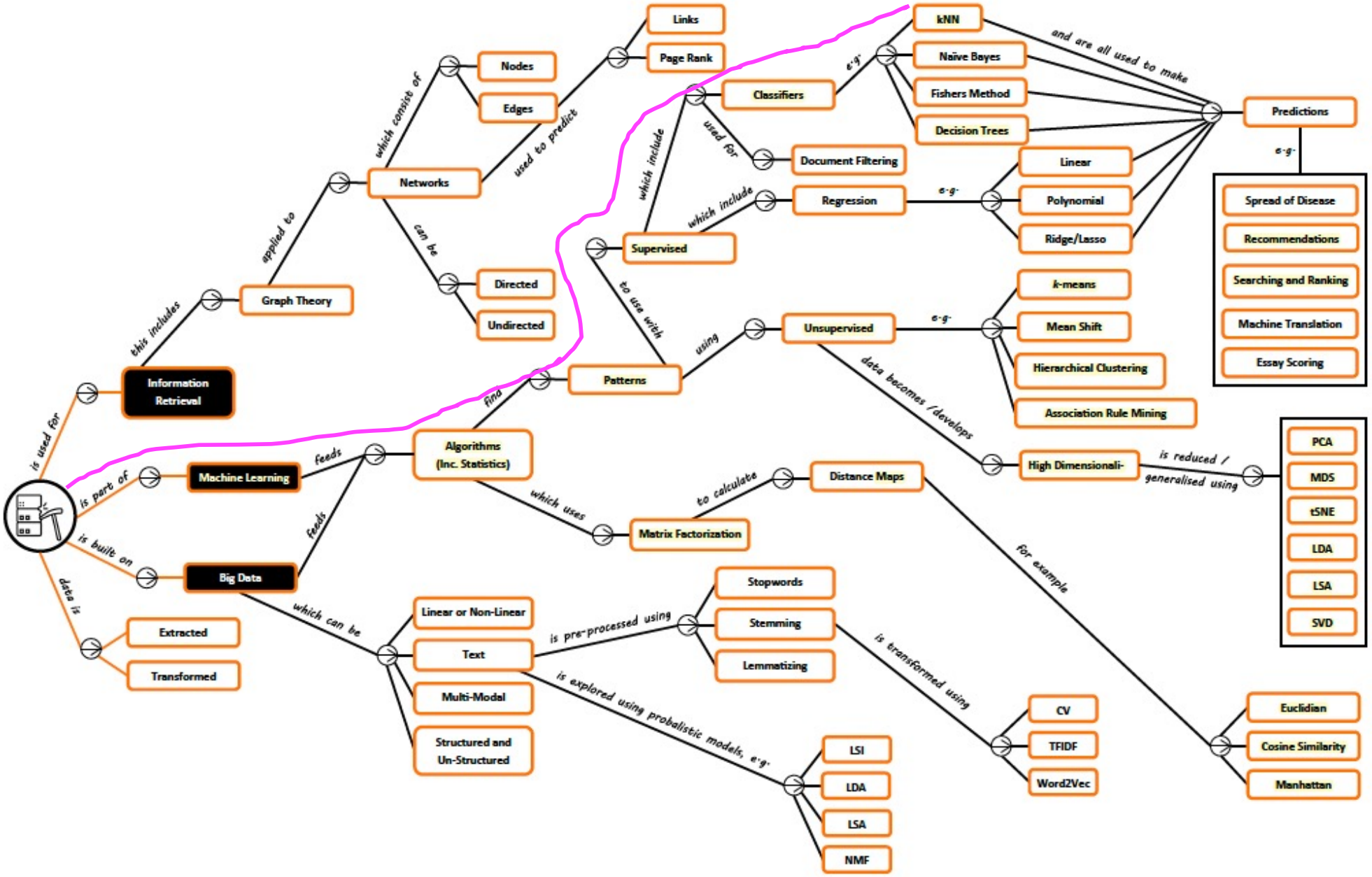
Gives low complexity , $\approx O(d \log n)$, as compare new data to only $\frac{n}{2^k}$

Demo: Nearest_Neighbours_Item_based

Nearest Neighbours – Summary

- Nearest neighbours can be used for **classification** and **regression**.
- Main limitations:
  - **Scales poorly** with large datasets (many items/users)
  - **Curse of dimensionality** in high-dimensional spaces
- Common mitigations:
  - Dimensionality reduction -> **Random Projection**
  - Efficient/approximate search -> **Hashing**

Nearest Neighbours – **Roadmap**



Nearest Neighbours – **Textbook**

University of Southampton

PART FOUR CLASSIFICATION

The classification task is to predict the label or class for a given unlabeled point. Formally, a classifier is a model or function M that predicts the class label $\hat{y}$ for a given input example $\mathbf{x}$, that is, $\hat{y} = M(\mathbf{x})$, where $\hat{y} \in \{c_1, c_2, \ldots, c_k\}$ and each c_i is a class label (a categorical attribute value). Classification is a *supervised learning* approach,

- Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

4 Classification: Alternative Techniques

The previous chapter introduced the classification problem and presented a technique known as the decision tree classifier. Issues such as model overfitting and model evaluation were also discussed. This chapter presents alternative techniques for building classification models—from simple techniques such as rule-based and nearest neighbor classifiers to more sophisticated techniques such as artificial neural networks, deep learning, support vector machines, and ensemble methods. Other practical issues such as the class imbalance and multiclass problems are also discussed at the end of the chapter.

- Introduction to Data Mining, *P. Tan et al*

Nearest Neighbours – Learning Outcomes

University of Southampton

- **LO1**: Demonstrate an understanding of the fundamentals of nearest neighbor classifiers, such as: (exam)
  - ❖ Understanding the k-NN algorithm, including key steps like distance calculations, voting, etc
  - ❖ Applying weighted k-NN and its use of weighted distance calculations
  - ❖ Discussing advantages and disadvantages of k-NN models
- **LO2**: Implement the learned algorithms using K-NN methods (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf*
- *ChatGPT or other AI-based techs*